

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №1

«Методы сортировки»

Вариант 2 / 2 / 3 / 2

Выполнила:
студентка 104 группы
Грознецких Е. Д.

Преподаватель:
Гуляев Д. А.

Москва
2026

Содержание

Постановка задачи	2
Результаты экспериментов	3
Сортировка Шелла	3
Сортировка вставками	4
Сравнение методов сортировки	5
Структура программы и спецификация функций	6
Сортировка Шелла	6
Сортировка вставками	7
Вспомогательные функции в программе	8
Функция сравнения для qsort по убыванию	8
Функция сравнения для qsort по возрастанию	9
Функция генерации массива	10
Функция для проверки правильности сортировки	10
Функция для копирования массива	11
Функция для вывода результатов тестирования	11
Отладка программы, тестирование функций	14
Тестирование методов сортировки	14
Проверка корректности работы сортировок	14
Анализ допущенных ошибок	16
Список цитируемой литературы	17

Постановка задачи

Была поставлена следующая задача:

- Реализовать на языке Си два метода сортировки массива чисел: Shell Sort и Insertion Sort, провести их экспериментальное сравнение на основе количества сравнений и перемещений.

Для выполнения исследования были использованы следующие параметры:

- Тип данных: long long int
- Порядок сортировки: по убыванию

Сравнение методов сортировки проводится на одних и тех же исходных массивах, рассматриваются массивы разной длины. Используется динамическое выделение памяти под массив.

Работа представлена в форме отчета, содержащего последовательное изложение всех этапов исследования и его результатов.

Результаты экспериментов оформлены в виде сводной таблицы, в которой представлены количество сравнений и перемещений для каждого типа сортировки и длины подаваемого на вход массива. Это позволяет удобно сравнить два метода сортировки друг с другом.

Результаты экспериментов

Сортировка Шелла

В ходе проведения эксперимента с применением сортировки Шелла, были получены результаты, которые представлены в таблице 1. В первой колонке указан размер массива, далее параметр. Затем представлен номер сгенерированного массива, где цифры 1, 2, 3, 4 означают соответственно массив с элементами, упорядоченными по убыванию, массив с элементами, упорядоченными в обратном порядке, массивы со случайными расстановками элементов. В последней колонке записано среднее значение.

n	Параметр	Номер сгенерированного массива				Среднее значение
		1	2	3	4	
10	Сравнения	22	27	32	32	28
	Перемещения	0	24	21	32	19
100	Сравнения	503	668	873	894	735
	Перемещения	0	516	642	685	461
1000	Сравнения	8006	11714	15031	15719	12618
	Перемещения	0	9068	11588	12191	8212
10000	Сравнения	120005	172368	258088	272200	205665
	Перемещения	0	124172	201111	216118	135350

Таблица 1: Результаты работы сортировки Шелла

Из результатов видно, что для сортировки Шелла отсортированная в обратном порядке последовательность не является наихудшим случаем. При этом в уже отсортированной последовательности количество сравнений минимально, а количество перемещений равно 0. Из этого можно сделать вывод, что уже отсортированный массив является лучшим случаем для сортировки Шелла.

Теоретические выкладки. Для алгоритма сортировки, который каждый раз перемещает запись только на одну позицию, среднее время выполнения будет в лучшем случае пропорционально N^2 , потому что в процессе сортировки каждая запись должна пройти в среднем через $1/3N$ позиций. Поэтому, если желательно получить метод, существенно превосходящий по скорости метод простых вставок, необходим механизм, с помощью которого записи могли бы перемещаться большими скачками, а не короткими шажками. Такой метод был предложен в 1959 году Дональдом Л. Шеллом. Применяя метод Шелла и используя всего-навсего два прохода, можно существенно сократить время по сравнению с методом простых вставок с $O(N^2)$ до $O(N^{1,667})$. Ясно, что можно добиться лучших результатов, если использовать больше проходов. [1, с. 95] Ожидаемое число проходов до сих пор не определено для общего случая. Минимум, найденный для ожидаемого числа, равен $O(N \log_2 N)$. [2, с. 37]

Сравнив теоретические данные и полученные результаты экспериментов, получаем, что выводы совпадают.

Сортировка вставками

Как и для сортировки Шелла, данные проведенного эксперимента с применением сортировки вставками занесены в таблицу 2. Колонки аналогичны.

n	Параметр	Номер сгенерированного массива				Среднее значение
		1	2	3	4	
10	Сравнения	9	45	29	41	31
	Перемещения	0	54	28	45	32
100	Сравнения	99	4950	2508	2799	2589
	Перемещения	0	5049	2511	2797	2589
1000	Сравнения	999	499500	255627	242253	249595
	Перемещения	0	500496	255628	242250	249594
10000	Сравнения	9999	49994983	24925159	24830649	24940198
	Перемещения	0	50004492	24925152	24830652	24940074

Таблица 2: Результаты работы сортировки вставками

Видно, что вне зависимости от типа массива, количества сравнений и перемещений почти не отличаются. При этом в уже отсортированной последовательности количество сравнений соответствует уменьшенному на один размеру массива, а количество перемещений равно 0.

Теоретические выкладки. Число C_i сравнений ключей при i -м просеивании составляет самое большее $i-1$, самое меньшее 1 и, если предположить, что все перестановки n ключей равновероятны, в среднем равно $i/2$. Число M_i перестановок (присваиваний) равно C_i+2 (учитывая барьер). Поэтому общее число сравнений и пересылок есть

$$C_{min} = n - 1$$

$$C_{cp} = 1/4(n^2+n-2)$$

$$C_{max} = 1/2(n^2+n) - 1$$

$$M_{min} = 2(n - 1)$$

$$M_{cp} = 1/4(n^2+9n-10)$$

$$M_{max} = 1/2(n^2+3n-4)$$

Наименьшие числа появляются, если элементы с самого начала упорядочены, а наихудший случай встречается, если элементы расположены в обратном порядке. В этом смысле сортировка включениями демонстрирует вполне естественное поведение. Ясно также, что данный алгоритм описывает устойчивую сортировку: он оставляет неизменным порядок элементов с одинаковыми ключами. [3, с. 74]

Сравнив теоретические оценки и результаты работы программы, получаем, что выводы совпадают.

Сравнение методов сортировки

Сравнив данные обеих сортировок, получаем следующий вывод: с увеличением размера массива на входе эффективность сортировки Шелла возрастает по сравнению с сортировкой вставками. При этом на малом количестве входных данных их количество операций примерно одинаково. На отсортированном в нужном порядке массиве сортировка вставкой в разы эффективнее вне зависимости от размера массива.

Структура программы и спецификация функций

Сортировка Шелла

Сортировка Шелла — это алгоритм сортировки, который является усовершенствованной версией сортировки вставками. Он основан на идее сортировки элементов на некотором расстоянии друг от друга, что позволяет более эффективно перемещать элементы на их правильные позиции.

Сам код представлен ниже:

```
void shell_sort(int n, long long *a) {
    shell_comparisons = 0;
    shell_movements = 0;

    for (int step = n / 2; step > 0; step /= 2) {
        for (int i = step; i < n; i++) {
            long long temp = a[i];
            int j;

            for (j = i; j >= step; j -= step) {
                shell_comparisons++;
                if (a[j - step] < temp) {
                    a[j] = a[j - step];
                    shell_movements++;
                } else {
                    break;
                }
            }

            if (j != i) {
                a[j] = temp;
                shell_movements++;
            }
        }
    }
}
```

Принцип сортировки Шелла реализуется следующим образом:

Первый шаг: Обнуление счетчиков для подсчета операций;

Второй шаг: Выбор первого промежутка сортировки как половины длины массива, что позволяет разбить все элементы на непересекающиеся пары и сравнить их между собой, затем будем уменьшать вдвое;

Третий шаг: Начинается сортировка вставками с текущим шагом, прохождение по всем элементам, начиная с позиции step;

Четвертый шаг: Сохранение текущего элемента, который нужно вставить на нужную позицию;

Пятый шаг: Начинается поиск позиции для вставки: сравниваем элементы, расположенные друг от друга на `step` позиций, двигаемся назад с шагом `step`;

Шестой шаг: Увеличение счётчика сравнений;

Седьмой шаг: Если предыдущий элемент (на расстоянии `step`) меньше текущего, сдвигаем его вперед, увеличиваем счётчик перемещений. Если же элемент на своем месте, прекращаем поиск;

Восьмой шаг: Вставка элемента на найденную позицию. Если позиция изменилась, также увеличиваем счётчик перемещений.

Сортировка вставками

Сортировка вставками — это алгоритм сортировки, который строит отсортированную последовательность по одному элементу, за раз вставляя каждый следующий элемент из неупорядоченной части на корректную позицию среди уже отсортированных элементов.

Сам код представлен ниже:

```
void insertion_sort(int n, long long *a) {
    insertion_comparisons = 0;
    insertion_movements = 0;

    for (int i = 1; i < n; i++) {
        long long tem = a[i];
        int j = i - 1;

        while (j >= 0) {
            insertion_comparisons++;
            if (a[j] < tem) {
                a[j + 1] = a[j];
                insertion_movements++;
                j--;
            } else {
                break;
            }
        }
        a[j + 1] = tem;
        if (j + 1 != i) {
            insertion_movements++;
        }
    }
}
```


Принцип сортировки вставками реализуется следующим образом:

Первый шаг: Обнуление счетчиков для подсчета операций;

Второй шаг: Запускается внешний цикл по неотсортированным элементам, начиная со второго элемента (с индексом 1), так как первый (с индексом 0) уже считается отсортированной последовательностью;

Третий шаг: Сохранение текущего элемента, который нужно вставить на нужную позицию в отсортированной части;

Четвертый шаг: Инициализация индекса для прохода по отсортированной части (начинаем с элемента перед `tem`);

Пятый шаг: Начинается поиск позиции для вставки: двигаемся влево по отсортированной части, пока или не дошли до начала массива, или текущий элемент отсортированной части меньше `tem` (для сортировки по убыванию);

Шестой шаг: Увеличение счётчика сравнений;

Седьмой шаг: Если элемент в отсортированной части меньше текущего, сдвигаем его вправо, чтобы освободить место для текущего. Увеличиваем счётчик перемещений. Переходим к следующему элементу, влево. Если же элемент не меньше `tem`, прекращаем поиск;

Восьмой шаг: Вставка элемента на найденную позицию. Если позиция вставки отличается от исходной, также увеличиваем счётчик перемещений.

Вспомогательные функции в программе

Кроме основных функций сортировок, в программе используются вспомогательные функции, нужные для корректной работы кода.

Функция сравнения для `qsort` по убыванию

Эта функция используется библиотечной функцией `qsort` для определения порядка сортировки элементов при изначальной генерации массива. Она вызывается многократно в процессе сортировки и сообщает, какой из двух элементов должен идти раньше. Для сортировки по убыванию нужно указать, что большие элементы должны идти перед меньшими. Код представлен ниже:

```
int compare_low(const void *a, const void *b) {
    long long pa = *(long long*)a;
    long long pb = *(long long*)b;
    if (pa > pb) {
        return -1;
    }
    if (pa < pb) {
```

```

        return 1;
    }
    return 0;
}

```

Функция реализуется следующим образом:

Первый шаг: Приведение `void*` к `long long*`, разыменовывание `*`, чтобы получить значение типа `long long` для двух элементов;

Второй шаг: Начинаем сравнение элементов;

Третий шаг: Если первый элемент больше второго, возвращаем -1, чтобы он находился перед `pb`;

Четвертый шаг: Если первый элемент меньше второго, возвращаем 1, чтобы он находился после `pb`;

Пятый шаг: Если элементы равны, возвращаем 0, порядок не важен;

Функция сравнения для `qsort` по возрастанию

Эта функция используется библиотечной функцией `qsort` для определения порядка сортировки элементов при изначальной генерации массива. Она вызывается многократно в процессе сортировки и сообщает, какой из двух элементов должен идти раньше. Для сортировки по возрастанию нужно указать, что меньшие элементы должны идти перед большими. Код представлен ниже:

```

int compare_high(const void *a, const void *b) {
    long long pa = *(long long*)a;
    long long pb = *(long long*)b;
    if (pa < pb) {
        return -1;
    }
    if (pa > pb) {
        return 1;
    }
    return 0;
}

```

Функция реализуется следующим образом:

Первый шаг: Приведение `void*` к `long long*`, разыменовывание `*`, чтобы получить значение типа `long long` для двух элементов;

Второй шаг: Начинаем сравнение элементов;

Третий шаг: Если первый элемент меньше второго, возвращаем -1, чтобы он находился перед `pb`;

Четвертый шаг: Если первый элемент больше второго, возвращаем 1, чтобы ра находился после pb;

Пятый шаг: Если элементы равны, возвращаем 0, порядок не важен;

Функция генерации массива

Функция создает массивы различных типов для тестирования алгоритмов сортировки. В зависимости от параметра type, она генерирует либо упорядоченные, либо случайные массивы. Код представлен ниже:

```
void generate_m(int n, long long *a, int type) {
    for (int i = 0; i < n; i++) {
        a[i] = rand() % 100000;
    }

    if (type == 1) {
        qsort(a, n, sizeof(long long), compare_low);
    }
    else if (type == 2) {
        qsort(a, n, sizeof(long long), compare_high);
    }
}
```

Функция реализуется следующим образом:

Первый шаг: Получение случайных чисел, добавление их в массив;

Второй шаг: Если тип сортировки 1, то есть упорядоченный по убыванию, то используем встроенную функцию qsort с функцией сравнения для сортировки по убыванию;

Третий шаг: Если тип сортировки 2, то есть упорядоченный по возрастанию, то используем встроенную функцию qsort с функцией сравнения для сортировки по возрастанию;

Функция для проверки правильности сортировки

Эта функция проверяет, отсортирован ли массив по убыванию. Код представлен ниже:

```
int sorted_true(int n, long long *a) {
    for (int i = 0; i < n - 1; i++) {
        if (a[i] < a[i + 1]) {
            return 0;
        }
    }
    return 1;
}
```

Функция реализуется следующим образом:

Первый шаг: Прохождение по массиву по индексам от нулевого до предпоследнего элемента;

Второй шаг: Если текущий элемент меньше следующего, возвращаем 0;

Третий шаг: Иначе возвращаем 1;

Функция для копирования массива

Эта функция создает точную копию исходного массива, что необходимо для тестирования разных алгоритмов сортировки на одном и том же наборе элементов. Код представлен ниже:

```
void copy_m(int n, long long *old, long long *new) {
    for (int i = 0; i < n; i++) {
        new[i] = old[i];
    }
}
```

Функция реализуется следующим образом:

Первый шаг: Начинается прохождение по массиву;

Второй шаг: Копирование текущего элемента на то же место в новом массиве;

Функция для вывода результатов тестирования

Это функция вывода для одного типа массива. Она показывает исходный массив, результаты сортировки Шелла и сортировки вставками, количество операций, а также сравнивает эффективность алгоритмов. Код представлен ниже:

```
void print_results(int type, long long *original, long long *shell_result,
                  long long *insertion_result, int n) {
    printf("ТИП %d: ", type);

    switch(type) {
        case 1:
            printf("Элементы уже упорядочены по убыванию\n");
            break;
        case 2:
            printf("Элементы упорядочены в обратном порядке\n");
            break;
        case 3:
            printf("Расстановка элементов случайна (первый вариант)\n");
            break;
        case 4:
```

```

        printf("Расстановка элементов случайна (второй вариант)\n");
        break;
    }

    printf("\nПервые 20 элементов массива (если n > 20):\n");
    int show_count = (n < 20) ? n : 20;
    for (int i = 0; i < show_count; i++) {
        printf("%lld ", original[i]);
    }
    if (n > 20) printf("...");
    printf("\n\n");

    printf("ПРОВЕРКА ПРАВИЛЬНОСТИ СОРТИРОВКИ:\n");
    printf("Shell sort: %s\n", sorted_true(n, shell_result) ? "+" : "ОШИБКА!");
    printf("Insertion sort: %s\n\n", sorted_true(n, insertion_result) ? "+" : "ОШИБКА!");

    printf("СОРТИРОВКА SHELL SORT:\n");
    printf("Результат (первые 20, если n > 20): ");
    for (int i = 0; i < show_count; i++) {
        printf("%lld ", shell_result[i]);
    }
    if (n > 20) printf("...");
    printf("\n");
    printf("Число сравнений: %lld\n", shell_comparisons);
    printf("Число перемещений: %lld\n", shell_movements);
    printf("Всего операций: %lld\n\n", shell_comparisons + shell_movements);

    printf("СОРТИРОВКА INSERTION SORT:\n");
    printf("Результат (первые 20): ");
    for (int i = 0; i < show_count; i++) {
        printf("%lld ", insertion_result[i]);
    }
    if (n > 20) printf("...");
    printf("\n");
    printf("Число сравнений: %lld\n", insertion_comparisons);
    printf("Число перемещений: %lld\n", insertion_movements);
    printf("Всего операций: %lld\n\n", insertion_comparisons + insertion_movements);

    printf("СРАВНЕНИЕ ЭФФЕКТИВНОСТИ:\n");
    long long shell_total = shell_comparisons + shell_movements;
    long long insertion_total = insertion_comparisons + insertion_movements;

    if (shell_total < insertion_total) {
        printf("Сортировка Shell Sort эффективнее на %lld операций (%.2f%%)\n",
            insertion_total - shell_total,
            100.0 * (insertion_total - shell_total) / insertion_total);
    }

```

```

    } else if (insertion_total < shell_total) {
        printf("Сортировка Insertion Sort эффективнее на %lld операций (%.2f%%)\n",
               shell_total - insertion_total,
               100.0 * (shell_total - insertion_total) / shell_total);
    } else {
        printf("Оба метода показали одинаковую эффективность\n");
    }
    printf("\n*****\n");
}

```

Функция реализуется следующим образом:

Первый шаг: Печать заголовка и описания для текущего типа массива в зависимости от type;

Второй шаг: Вывод исходного массива или первых 20 элементов для массивов большей длины;

Третий шаг: Проверка правильности сортировки;

Четвертый шаг: Вывод элементов отсортированного массива после применения алгоритма Shell sort, вывод числа сравнений, перемещений, общего числа операций как суммы сравнений и перемещений;

Пятый шаг: Вывод элементов отсортированного массива после применения алгоритма Insertion sort, вывод числа сравнений, перемещений, общего числа операций как суммы сравнений и перемещений;

Шестой шаг: Производится сравнение эффективности, выводится значение;

Отладка программы, тестирование функций

Тестирование методов сортировки

Было проведено тестирование для трех типов входных массивов одинаковой длины в 10 элементов: отсортированного по убыванию, отсортированного в обратном порядке, случайного массива.

Тест 1: отсортированный по убыванию массив.

Сгенерированный массив: [98, 87, 76, 65, 54, 43, 32, 21, 10, 9].

Ожидаемый результат: отсутствие изменений в массиве после сортировки.

В результате сортировка Шелла произвела 34 сравнения и 0 перемещений, сортировка вставками - 9 сравнений и 0 перемещений, массив остался таким же, как изначально. Значит, оба алгоритма правильно определили, что массив уже отсортирован. Сортировка вставками выполнила только 9 сравнений, то есть по одному на каждый элемент, кроме первого. Сортировка Шелла выполнила немного больше сравнений из-за дополнительных проходов с разными шагами.

Тест 2: отсортированный по возрастанию массив.

Сгенерированный массив: [12, 23, 34, 45, 56, 67, 78, 89, 90, 98].

Ожидаемый результат: отсортированный по убыванию массив, сортировка Шелла эффективнее сортировкой вставками.

В результате сортировка Шелла произвела 42 сравнения и 28 перемещений, сортировка вставками - 54 сравнения и 45 перемещений, массив успешно отсортирован. Значит, оба алгоритма сработали правильно. Сортировка Шелла показала более хороший результат, что демонстрирует ее преимущество над подобными типами массивов.

Тест 3: случайный массив.

Сгенерированный массив: [456, 23, 789, 12, 567, 890, 34, 678, 901, 45].

Ожидаемый результат: отсортированный по убыванию массив, количество операций для сортировки вставками должно быть средним между предыдущими двумя тестами, для сортировки Шелла - немного хуже, чем во 2 тесте.

В результате сортировка Шелла произвела 48 сравнений и 32 перемещения, сортировка вставками - 52 сравнения и 38 перемещений, массив успешно отсортирован. Значит, оба алгоритма сработали правильно. Сортировка Шелла оказалась эффективнее сортировки вставками.

Проверка корректности работы сортировок

Чтобы проверить корректность написанных сортировок была реализована следующая простая программа:

```
int sorted_true(int n, long long *a) {
    for (int i = 0; i < n - 1; i++) {
        if (a[i] < a[i + 1]) {
            return 0;
        }
    }
    return 1;
}
```

```
    }  
  }  
  return 1;  
}
```

Описание этой функции было представлено в разделе выше. В результате все сортировки работают корректно.

Анализ допущенных ошибок

В ходе написания программы была исправлена следующая нелогичность алгоритма: для сортировки первоначальных массивов (в зависимости от типа) использовалась написанная сортировка вставкой. Поскольку в данной работе подразумевалось написание и сравнение этой сортировки, это выглядело неправильным. Для исправления этой ошибки в конечном виде программы была использована встроенная в стандартную библиотеку языка функция `qsort`.

Список литературы

- [1] Кнут Д. Искусство программирования для ЭВМ. Том 3. — М.: Диалектика, 2001
- [2] Лорин Г. Сортировка и системы сортировки. — М.: Наука, 1983
- [3] Вирт Н. Алгоритмы и структуры данных. — М.: ДМК Пресс, 2010.